IVS Projekt- Kalkulačka

Generated by Doxygen 1.9.8

1 File Index 1

1.1 File List . . . 1

2 File Documentation 3

2.1 gui.c File Reference . . . 3

2.1.1 Detailed Description . . . 4

2.1.2 Function Documentation . . . 4

2.1.2.1 main() . . . 4

2.1.2.2 on_backspace_clicked() . . . 4

2.1.2.3 on_button_clicked() . . . 4

2.1.2.4 on_clear_clicked() . . . 5

2.1.2.5 on_key_press() . . . 5

2.1.2.6 run_calculator() . . . 5

2.1.3 Variable Documentation . . . 5

2.1.3.1 err_msg . . . 5

2.2 gui.h File Reference . . . 6

2.2.1 Detailed Description . . . 7

2.2.2 Function Documentation . . . 7

2.2.2.1 on_backspace_clicked() . . . 7

2.2.2.2 on_button_clicked() . . . 7

2.2.2.3 on_clear_clicked() . . . 8

2.2.2.4 on_key_press() . . . 8

2.2.2.5 run_calculator() . . . 8

2.3 gui.h . . . 9

2.4 math.c File Reference . . . 9

2.4.1 Detailed Description . . . 10

2.4.2 Function Documentation . . . 10

2.4.2.1 add() . . . 10

2.4.2.2 divide() . . . 11

2.4.2.3 factorial() . . . 11

2.4.2.4 logarithm() . . . 11

2.4.2.5 mul() . . . 12

2.4.2.6 power() . . . 12

2.4.2.7 root() . . . 12

2.4.2.8 sub() . . . 13

2.5 math.h File Reference . . . 13

2.5.1 Detailed Description . . . 14

2.5.2 Function Documentation . . . 14

2.5.2.1 add() . . . 14

2.5.2.2 divide() . . . 15

2.5.2.3 factorial() . . . 15

2.5.2.4 logarithm() . . . 15

2.5.2.5 mul() . . . 16
2.5.2.6 n_root() . . . 16
2.5.2.7 power() . . . 17
2.5.2.8 sub() . . . 17
2.6 math.h . . . 17
2.7 parser.c File Reference . . . 18
2.7.1 Detailed Description . . . 19
2.7.2 Function Documentation . . . 20
2.7.2.1 parse_add() . . . 20
2.7.2.2 parse_div() . . . 20
2.7.2.3 parse_equation() . . . 20
2.7.2.4 parse_factorial() . . . 21
2.7.2.5 parse_log() . . . 21
2.7.2.6 parse_mul() . . . 22
2.7.2.7 parse_number() . . . 22
2.7.2.8 parse_power() . . . 23
2.7.2.9 parse_root() . . . 23
2.7.2.10 parse_sub() . . . 23
2.8 parser.c . . . 24
2.9 parser.h File Reference . . . 28
2.9.1 Detailed Description . . . 30
2.9.2 Function Documentation . . . 30
2.9.2.1 parse_add() . . . 30
2.9.2.2 parse_div() . . . 30
2.9.2.3 parse_equation() . . . 31
2.9.2.4 parse_factorial() . . . 31
2.9.2.5 parse_log() . . . 32
2.9.2.6 parse_mul() . . . 32
2.9.2.7 parse_number() . . . 33
2.9.2.8 parse_power() . . . 33
2.9.2.9 parse_root() . . . 33
2.9.2.10 parse_sub() . . . 34
2.10 parser.h . . . 34
2.11 test_math.c File Reference . . . 35
2.11.1 Function Documentation . . . 36
2.11.1.1 absolute() . . . 36
2.11.1.2 add() . . . 36
2.11.1.3 divide() . . . 36
2.11.1.4 factorial() . . . 36
2.11.1.5 is_equal() . . . 36
2.11.1.6 logarithm() . . . 36
2.11.1.7 main() . . . 36

2.11.1.8 mul() 36
2.11.1.9 n_root() 37
2.11.1.10 power() 37
2.11.1.11 sub() 37

Index **39**

Chapter 1

File Index

1.1 File List

Here is a list of all files with brief descriptions:

gui.c
Graphical user interface of the calculator app 3
gui.h
Header file for the graphical user interface of the calculator app 6
math.c
Library for calculting math functions 9
math.h
Library for calculating mathematical functions 13
parser.c
Parser of the calculator app 18
parser.h
Parser of the calculator app 28
test_math.c 35

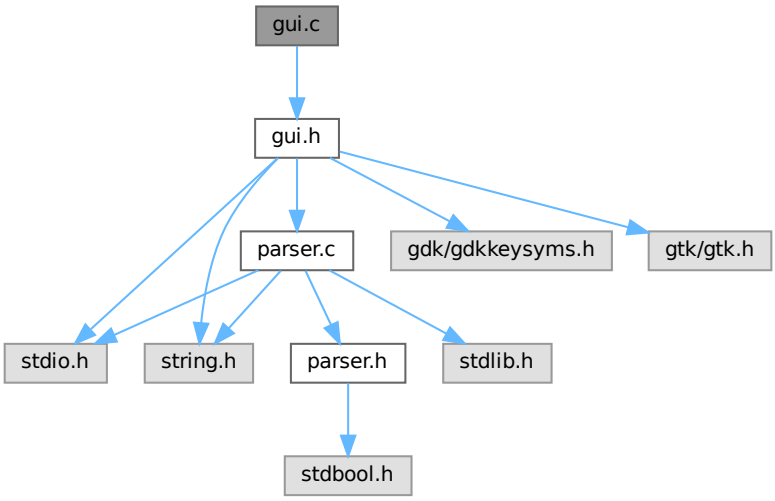
Chapter 2

File Documentation

2.1 gui.c File Reference

graphical user interface of the calculator app

`#include "gui.h"`
Include dependency graph for gui.c:



Functions

- void on_button_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Handles button clicks (numbers and operators)
- void on_clear_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Clears the display.
- void on_backspace_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Deletes the last character from the display.

- gboolean on_key_press (GtkWidget ∗window, GdkEventKey ∗event, GtkWidget ∗display)

  Deletes the last character from the display.
- int run_calculator (int argc, char ∗argv[ ])

  Initializes and runs the calculator GUI.
- int main (int argc, char ∗argv[ ])

Variables

- const char ∗ err_msg [ ]

2.1.1 Detailed Description

graphical user interface of the calculator app

Author

Daniel Kratky

2.1.2 Function Documentation

2.1.2.1 main()

```
int main (
            int argc,
            char * argv[] )
```

Program entry point

2.1.2.2 on_backspace_clicked()

```
void on_backspace_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Deletes the last character from the display.

Deletes last character on the display

2.1.2.3 on_button_clicked()

```
void on_button_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Handles button clicks (numbers and operators)

Handles button clicks (numbers and operators)

2.1.2.4 on_clear_clicked()

```
void on_clear_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Clears the display.

Clears the display

2.1.2.5 on_key_press()

```
gboolean on_key_press (
            GtkWidget * window,
            GdkEventKey * event,
            GtkWidget * display )
```

Deletes the last character from the display.

Removes the last character in the entry field if it exists.

Parameters

window	Pointer to the window widget
event	Pointer to the event that triggered the signal
display	Pointer to the display entry all three parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

2.1.2.6 run_calculator()

```
int run_calculator (
            int argc,
            char * argv[] )
```

Initializes and runs the calculator GUI.

Initializes and runs the calculator GUI

2.1.3 Variable Documentation

2.1.3.1 err_msg

```
const char* err_msg[]
```

Initial value:

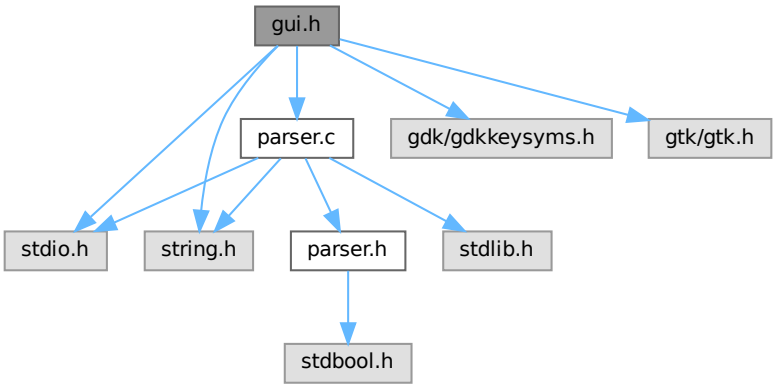
```
= {
    "Error: Multimle decimal points",
    "Error: Unrecognized character",
    "Error: No input for function",
    "Error: Number out of bounds",
}
```

2.2 gui.h File Reference

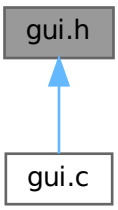
Header file for the graphical user interface of the calculator app.

```
#include <stdio.h>
#include <string.h>
#include <gdk/gdkkeysyms.h>
#include <gtk/gtk.h>
#include "parser.c"
```

Include dependency graph for gui.h:



This graph shows which files directly or indirectly include this file:



Functions

- void on_button_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Handles button clicks (numbers and operators)
- void on_clear_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Clears the display.
- void on_backspace_clicked (GtkWidget ∗button, GtkWidget ∗display)

  Deletes the last character from the display.

- gboolean on_key_press (GtkWidget *window, GdkEventKey *event, GtkWidget *display)

  Deletes the last character from the display.
- int run_calculator (int argc, char *argv[ ])

  Initializes and runs the calculator GUI.

2.2.1 Detailed Description

Header file for the graphical user interface of the calculator app.

Author

Daniel Kratky

This file declares the global variables and function prototypes used in the calculator program. The calculator provides a basic graphical user interface (GUI) with buttons and a display field

2.2.2 Function Documentation

2.2.2.1 on_backspace_clicked()

```
void on_backspace_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Deletes the last character from the display.

Removes the last character in the entry field if it exists.

Parameters

button	Pointer to the button widget
display	Pointer to the display entry both parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

Deletes last character on the display

2.2.2.2 on_button_clicked()

```
void on_button_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Handles button clicks (numbers and operators)

Appends the pressed button's label to the display.

Parameters

button	Pointer to the button widget
display	Pointer to the display entry both parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

Handles button clicks (numbers and operators)

2.2.2.3 on_clear_clicked()

```
void on_clear_clicked (
            GtkWidget * button,
            GtkWidget * display )
```

Clears the display.

Sets the display text to an empty string.

Parameters

button	Pointer to the button widget
display	Pointer to the display entry both parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

Clears the display

2.2.2.4 on_key_press()

```
gboolean on_key_press (
            GtkWidget * window,
            GdkEventKey * event,
            GtkWidget * display )
```

Deletes the last character from the display.

Removes the last character in the entry field if it exists.

Parameters

window	Pointer to the window widget
event	Pointer to the event that triggered the signal
display	Pointer to the display entry all three parameters are necessary to folow the gtk conventions and work with other functions they can accept null pointers if unused

2.2.2.5 run_calculator()

```
int run_calculator (
            int argc,
            char * argv[] )
```

Initializes and runs the calculator GUI.

Creates the window, sets up all widgets, and starts the GTK main event loop.

Parameters

argc	Argument count from main()
argv	Argument vector from main()

Returns

int Exit status

Initializes and runs the calculator GUI

2.3 gui.h

Go to the documentation of this file.

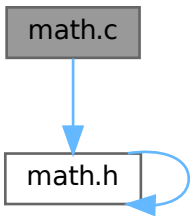
```
00001 //======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
00002 //
00003 // Purpose:     calculator GUI
00004 //
00005 // $NoKeywords: $ivs_project_2 $gui.h
00006 // $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
00007 // $Date:       $2026-04-23
00008 //===========================================================================//
00009 #ifndef GUI_H
00010 #define GUI_H
00011
00023 #include <stdio.h>
00024 #include <string.h>
00025 #include <gdk/gdkkeysyms.h>
00026 #include <gtk/gtk.h>
00027 #include "parser.c"
00028
00039 void on_button_clicked(GtkWidget *button, GtkWidget *display);
00040
00051 void on_clear_clicked(GtkWidget *button, GtkWidget *display);
00052
00063 void on_backspace_clicked(GtkWidget *button, GtkWidget *display);
00064
00076 gboolean on_key_press(GtkWidget *window, GdkEventKey *event, GtkWidget *display);
00077
00088 int run_calculator(int argc, char *argv[]);
00089
00090 #endif // GUI_H
```

2.4 math.c File Reference

Library for calculting math functions.

`#include "math.h"`

Include dependency graph for math.c:



Functions

- double add (double a, double b, int ∗error)

  Adds two numbers.
- double sub (double a, double b, int ∗error)

  Subtracts two numbers.
- double mul (double a, double b, int ∗error)

  Multiplies two numbers.
- double divide (double a, double b, int ∗error)

  Divides two numbers.
- long long factorial (int n, int ∗error)

  Computes factorial of a number.
- double power (double base, int exponent, int ∗error)

  Computes power function.
- double root (int degree, double radicand, int ∗error)
- double logarithm (double x, int ∗error)

  Computes base-10 logarithm.

2.4.1 Detailed Description

Library for calculting math functions.

Author

Daniel Kratky, Radek Zak

2.4.2 Function Documentation

2.4.2.1 add()

```
double add (
            double a,
            double b,
            int * error )
```

Adds two numbers.

Parameters

a	First addend
b	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

Sum of a and b

2.4.2.2 divide()

```
double divide (
            double a,
            double b,
            int * error )
```

Divides two numbers.

Parameters

a	Numerator
b	Denominator
error	Pointer to error code (0 if no error occurred)

Returns

Result of division (a / b)

2.4.2.3 factorial()

```
long long factorial (
            int n,
            int * error )
```

Computes factorial of a number.

Parameters

n	Non-negative integer
error	Pointer to error code (0 if no error occurred)

Returns

n!

2.4.2.4 logarithm()

```
double logarithm (
            double x,
            int * error )
```

Computes base-10 logarithm.

Parameters

x	Input value (must be $>$ 0)
error	Pointer to error code (0 if no error occurred)

Returns

log10(x)

2.4.2.5 mul()

```
double mul (
            double a,
            double b,
            int * error )
```

Multiplies two numbers.

Parameters

a	First factor
b	Second factor
error	Pointer to error code (0 if no error occurred)

Returns

Product of a and b

2.4.2.6 power()

```
double power (
            double base,
            int exponent,
            int * error )
```

Computes power function.

Parameters

base	Base value
exponent	Integer exponent
error	Pointer to error code (0 if no error occurred)

Returns

base raised to the power exponent

2.4.2.7 root()

```
double root (
            int degree,
            double radicand,
            int * error )
```

2.4.2.8 sub()

```
double sub (
            double a,
            double b,
            int * error )
```

Subtracts two numbers.

Parameters

a	Minuend
b	Subtrahend
error	Pointer to error code (0 if no error occurred)

Returns

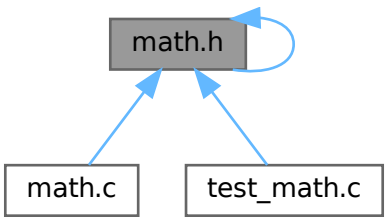
Difference (a - b)

2.5 math.h File Reference

Library for calculating mathematical functions.

`#include <math.h>`
Include dependency graph for math.h:

This graph shows which files directly or indirectly include this file:



Functions

- double add (double a, double b, int ∗error)

  Adds two numbers.
- double sub (double a, double b, int ∗error)

  Subtracts two numbers.
- double mul (double a, double b, int ∗error)

  Multiplies two numbers.
- double divide (double a, double b, int ∗error)

  Divides two numbers.
- long long factorial (int n, int ∗error)

  Computes factorial of a number.
- double power (double base, int exponent, int ∗error)

  Computes power function.
- double n_root (int degree, double radicand, int ∗error)

  Computes n-th root of a number.
- double logarithm (double x, int ∗error)

  Computes base-10 logarithm.

2.5.1 Detailed Description

Library for calculating mathematical functions.

Author

Daniel Kratky

This file declares function prototypes for basic arithmetic and advanced mathematical operations used in the calculator.

2.5.2 Function Documentation

2.5.2.1 add()

```
double add (
            double a,
            double b,
            int * error )
```

Adds two numbers.

Parameters

a	First addend
b	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

Sum of a and b

2.5.2.2 divide()

```
double divide (
            double a,
            double b,
            int * error )
```

Divides two numbers.

Parameters

a	Numerator
b	Denominator
error	Pointer to error code (0 if no error occurred)

Returns

Result of division (a / b)

2.5.2.3 factorial()

```
long long factorial (
            int n,
            int * error )
```

Computes factorial of a number.

Parameters

n	Non-negative integer
error	Pointer to error code (0 if no error occurred)

Returns

n!

2.5.2.4 logarithm()

```
double logarithm (
            double x,
            int * error )
```

Computes base-10 logarithm.

Parameters

x	Input value (must be $>$ 0)
error	Pointer to error code (0 if no error occurred)

Returns

log10(x)

2.5.2.5 mul()

```
double mul (
            double a,
            double b,
            int * error )
```

Multiplies two numbers.

Parameters

a	First factor
b	Second factor
error	Pointer to error code (0 if no error occurred)

Returns

Product of a and b

2.5.2.6 n_root()

```
double n_root (
            int degree,
            double radicand,
            int * error )
```

Computes n-th root of a number.

Parameters

degree	Root degree (e.g., 2 for square root)
radicand	Value under the root
error	Pointer to error code (0 if no error occurred)

Returns

n-th root of radicand

2.5.2.7 power()

```
double power (
            double base,
            int exponent,
            int * error )
```

Computes power function.

Parameters

base	Base value
exponent	Integer exponent
error	Pointer to error code (0 if no error occurred)

Returns

base raised to the power exponent

2.5.2.8 sub()

```
double sub (
            double a,
            double b,
            int * error )
```

Subtracts two numbers.

Parameters

a	Minuend
b	Subtrahend
error	Pointer to error code (0 if no error occurred)

Returns

Difference (a - b)

2.6 math.h

Go to the documentation of this file.

```
00001 //======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
00002 //
00003 // Purpose:     math library
00004 //
00005 // $NoKeywords: $ivs_project_2 $math.h
00006 // $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
00007 // $Date:       $2026-04-24
00008 //===========================================================================//
00009 #ifndef MATH_H
00010 #define MATH_H
00021 #include <math.h>
00022
```

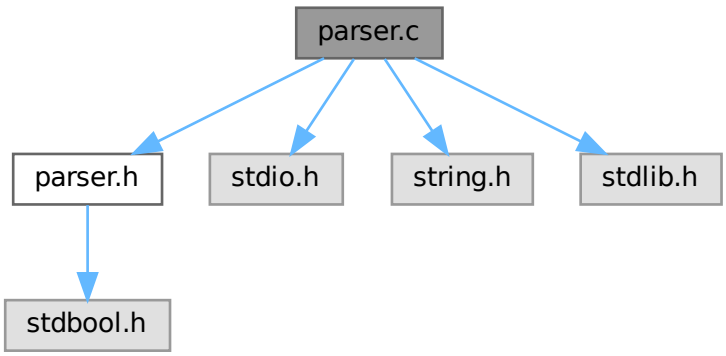
```
double add(double a, double b, int *error);

double sub(double a, double b, int *error);

double mul(double a, double b, int *error);

double divide(double a, double b, int *error);

long long factorial(int n, int *error);

double power(double base, int exponent, int *error);

double n_root(int degree, double radicand, int *error);

double logarithm(double x, int *error);

#endif // MATH_H
```

2.7 parser.c File Reference

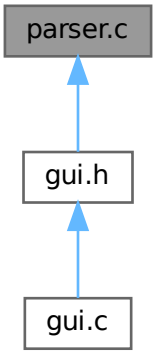
parser of the calculator app

```
#include "parser.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

Include dependency graph for parser.c:



This graph shows which files directly or indirectly include this file:



Functions

- double parse_factorial (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for factorials and evaluates them
- double parse_power (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for powers and evaluates them
- double parse_root (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for roots and evaluates them
- double parse_log (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for logarithms and evaluates them
- double parse_mul (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for multiplications and evaluates them
- double parse_div (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for divisions and evaluates them
- double parse_add (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for additions and evaluates them
- double parse_sub (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for subtractions and evaluates them
- double parse_equation (const char ∗equation, unsigned int lenght, unsigned int depth, unsigned int ∗error)

  parses a string into a mathematical equation and returns its value
- double parse_number (const char ∗number, unsigned int lenght, unsigned int ∗error)

  parses a string into decimal number

2.7.1 Detailed Description

parser of the calculator app

Author

Daniel Kratky

2.7.2 Function Documentation

2.7.2.1 parse_add()

```
double parse_add (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for additions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.7.2.2 parse_div()

```
double parse_div (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for divisions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.7.2.3 parse_equation()

```
double parse_equation (
            const char * equation,
```

```
            unsigned int lenght,
            unsigned int depth,
            unsigned int * error )
```

parses a string into a mathematical equation and returns its value

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

value of the equation

2.7.2.4 parse_factorial()

```
double parse_factorial (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for factorials and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.7.2.5 parse_log()

```
double parse_log (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for logarithms and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.7.2.6 parse_mul()

```
double parse_mul (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for multiplications and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.7.2.7 parse_number()

```
double parse_number (
            const char * number,
            unsigned int lenght,
            unsigned int * error )
```

parses a string into decimal number

Parameters

number	Pointer to the string containing the number
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

decimal value of the number

2.7.2.8 parse_power()

```
double parse_power (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for powers and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.7.2.9 parse_root()

```
double parse_root (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for roots and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.7.2.10 parse_sub()

```
double parse_sub (
            const char * equation,
```

```
unsigned int lenght,
unsigned int * error,
bool * parsed )
```

searches the string equation for subtractions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.8 parser.c

Go to the documentation of this file.

```
00001 //======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
00002 //
00003 // Purpose:     calculator parser
00004 //
00005 // $NoKeywords: $ivs_project_2 $parser.c
00006 // $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
00007 // $Date:       $2026-04-28
00008 //===========================================================================//
00015 #include "parser.h"
00016 #include <stdio.h>
00017 #include <string.h>
00018 #include <stdlib.h>
00019
00020
00021 double parse_factorial(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
00022     for (int character = lenght - 1; character >= 0; character--) {
00023         if (equation[character] == '!') {
00024             *parsed = true;
00025
00026             // if there is nothing as input
00027             if (character == 0) {
00028                 *error = 3;
00029                 return 0.0;
00030             }
00031
00032             double num_l = parse_equation(equation, character, 0, error);
00033             if(*error) {
00034                 return 0.0;
00035             }
00036
00037             // TODO: change for factorial(num_r, error); once implemented
00038             return 1.0 + num_l;
00039
00040         }
00041     }
00042     *parsed = false;
00043     return 0.0;
00044 }
00045
00046 double parse_power(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
00047     for (int character = lenght - 1; character >= 0; character--) {
00048         if (equation[character] == '^') {
00049             *parsed = true;
00050
00051             const char *equation_r = equation + character + 1;
00052             const char *equation_l = equation;
00053             unsigned int lenght_r = lenght - character - 1;
00054             unsigned int lenght_l = character;
00055
00056             // if there is nothing as input
00057             if (lenght_r == 0 || lenght_l == 0) {
```

```
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 1, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 2, error);
            if(*error) {
                return 0.0;
            }

            // TODO: change for power(num_l, num_r, error); once implemented
            return 1.0 + num_r + num_l;
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_root(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == 'r') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 2, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 3, error);
            if(*error) {
                return 0.0;
            }

            // TODO: change for root(num_r, error); once implemented
            return 1.0 + num_r + num_l;
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_log(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == 'l') {
            *parsed = true;
            const char *equation_r = equation + character + 3;
            unsigned int lenght_r = lenght - character - 3;
            if (lenght_r == 0) {
                *error = 3;
                return 0.0;
            }
            double num_r = parse_equation(equation_r, lenght_r, 3, error);
            if(*error) {
                return 0.0;
            }
            // TODO: change for log(num_r, error); once implemented
            return 1.0 + num_r;
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_mul(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '*') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;
```

```

            // if there is nothing as input
            if (lenght_r == 0 || lenght_l == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 4, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 5, error);
            if(*error) {
                return 0.0;
            }

            // TODO: change for mul(num_l, num_r, error); once implemented
            return 1.0 + num_r + num_l;
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_div(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '/') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0 || lenght_l == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 5, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 6, error);
            if(*error) {
                return 0.0;
            }

            // TODO: change for div(num_l, num_r, error); once implemented
            return 1.0 + num_r + num_l;
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_add(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '+') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0 || lenght_l == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 6, error);
            if(*error) {
                return 0.0;
            }

            double num_l = parse_equation(equation_l, lenght_l, 7, error);
            if(*error) {
                return 0.0;
            }

            // TODO: change for add(num_l, num_r, error); once implemented
            return 1.0 + num_r + num_l;
```

```
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_sub(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed) {
    for (int character = lenght - 1; character >= 0; character--) {
        if (equation[character] == '-') {
            *parsed = true;

            const char *equation_r = equation + character + 1;
            const char *equation_l = equation;
            unsigned int lenght_r = lenght - character - 1;
            unsigned int lenght_l = character;

            // if there is nothing as input
            if (lenght_r == 0) {
                *error = 3;
                return 0.0;
            }

            double num_r = parse_equation(equation_r, lenght_r, 7, error);
            if(*error) {
                return 0.0;
            }
            // parsing possible negative numbers
            double num_l;
            if (lenght_l == 0) {
                num_l = 0.0;
            } else {
                num_l = parse_equation(equation_l, lenght_l, 7, error);
            }
            if(*error) {
                return 0.0;
            }

            // TODO: change for sub(num_l, num_r, error); once implemented
            return 1.0 + num_r + num_l;
        }
    }
    *parsed = false;
    return 0.0;
}

double parse_equation(const char *equation, unsigned int lenght, unsigned int depth, unsigned int
*error) {
    bool parsed;
    double value;
    // parses for each operator, ignores the one for which it already checked
    switch(depth) {
        case 0:
            value = parse_factorial(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 1:
            value = parse_power(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 2:
            value = parse_root(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 3:
            value = parse_log(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 4:
            value = parse_mul(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 5:
            value = parse_div(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 6:
            value = parse_add(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        case 7:
```

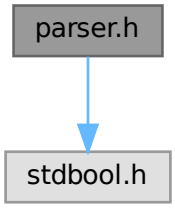
```
            value = parse_sub(equation, lenght, error, &parsed);
            if (parsed) {
                return value;
            }
        default:
            return parse_number(equation, lenght, error);
    }
}

double parse_number(const char *number, unsigned int lenght, unsigned int *error) {
    int decimal_points = 0;
    for (unsigned int character = 0; character < lenght; character++) {
        // Catch multiple decimals in one number
        if (number[character] == '.') {
            decimal_points++;
        }
        // Catch nonstandard characters
        if (!(number[character] == '.' ||
              number[character] == '0' ||
              number[character] == '1' ||
              number[character] == '2' ||
              number[character] == '3' ||
              number[character] == '4' ||
              number[character] == '5' ||
              number[character] == '6' ||
              number[character] == '7' ||
              number[character] == '8' ||
              number[character] == '9')) {
            *error = 2;
            return 0.0;
        }
    }
    if (decimal_points > 1) {
        *error = 1;
        return 0.0;
    }
    return strtod(number, NULL);
}

/*int main(int argc, char *argv[]) {
    nuber test
    unsigned int error1 = 0;
    unsigned int error2 = 0;
    unsigned int error3 = 0;
    unsigned int error4 = 0;
    char *number1 = "123";
    char *number2 = "123.00";
    char *number3 = "123.0.";
    char *number4 = "12a3";
    double num1 = parse_number(number1, 3, &error1);
    double num2 = parse_number(number2, 6, &error2);
    double num3 = parse_number(number3, 6, &error3);
    double num4 = parse_number(number4, 4, &error4);
    printf("%f Error: %i \n", num1, error1);
    printf("%f Error: %i \n", num2, error2);
    printf("%f Error: %i \n", num3, error3);
    printf("%f Error: %i \n", num4, error4);
    unsigned int error1 = 0;
    char *equation1 = "123-7*12.5+-14/11r3^2!";
    double num1 = parse_equation(equation1, 22, 0, &error1);
    printf("%f Error: %i \n", num1, error1);
    return 0; //
}*/
```

2.9 parser.h File Reference

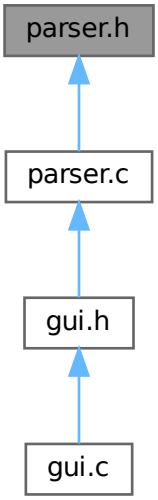
parser of the calculator app

```
#include <stdbool.h>
```

Include dependency graph for parser.h:



This graph shows which files directly or indirectly include this file:



Functions

- double parse_factorial (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for factorials and evaluates them
- double parse_power (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for powers and evaluates them
- double parse_root (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for roots and evaluates them
- double parse_log (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for logarithms and evaluates them
- double parse_mul (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

searches the string equation for multiplications and evaluates them

- double parse_div (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for divisions and evaluates them
- double parse_add (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for additions and evaluates them
- double parse_sub (const char ∗equation, unsigned int lenght, unsigned int ∗error, bool ∗parsed)

  searches the string equation for subtractions and evaluates them
- double parse_equation (const char ∗equation, unsigned int lenght, unsigned int depth, unsigned int ∗error)

  parses a string into a mathematical equation and returns its value
- double parse_number (const char ∗number, unsigned int lenght, unsigned int ∗error)

  parses a string into decimal number

2.9.1 Detailed Description

parser of the calculator app

Author

Daniel Kratky

This file declares function prototypes for functions that handle user input

2.9.2 Function Documentation

2.9.2.1 parse_add()

```
double parse_add (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for additions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9.2.2 parse_div()

```
double parse_div (
            const char * equation,
```

```
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for divisions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9.2.3 parse_equation()

```
double parse_equation (
            const char * equation,
            unsigned int lenght,
            unsigned int depth,
            unsigned int * error )
```

parses a string into a mathematical equation and returns its value

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

value of the equation

2.9.2.4 parse_factorial()

```
double parse_factorial (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for factorials and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9.2.5 parse_log()

```
double parse_log (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for logarithms and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9.2.6 parse_mul()

```
double parse_mul (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for multiplications and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9.2.7 parse_number()

```
double parse_number (
            const char * number,
            unsigned int lenght,
            unsigned int * error )
```

parses a string into decimal number

Parameters

number	Pointer to the string containing the number
lenght	Lenght of the string containing the equation
depth	Second addend
error	Pointer to error code (0 if no error occurred)

Returns

decimal value of the number

2.9.2.8 parse_power()

```
double parse_power (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for powers and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9.2.9 parse_root()

```
double parse_root (
            const char * equation,
```

```
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for roots and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.9.2.10 parse_sub()

```
double parse_sub (
            const char * equation,
            unsigned int lenght,
            unsigned int * error,
            bool * parsed )
```

searches the string equation for subtractions and evaluates them

Parameters

equation	Pointer to the string containing the equation
lenght	Lenght of the string containing the equation
error	Pointer to error code (0 if no error occurred)
parsed	Pointer bool set to true if the function parsed anythng, false otherwise

Returns

value of the equation

2.10 parser.h

Go to the documentation of this file.

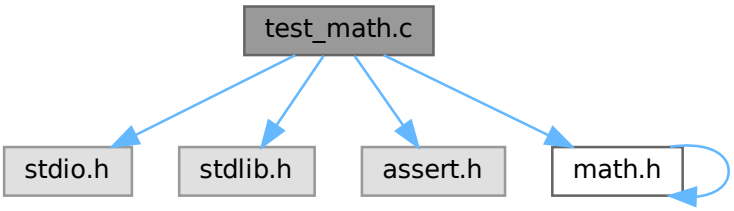
```
00001 //======= Copyright (c) 2026, FIT VUT Brno, GNU GPL V3.0 ============//
00002 //
00003 // Purpose:     calculator parser
00004 //
00005 // $NoKeywords: $ivs_project_2 $parser.h
00006 // $Authors:    Daniel Kratky <xkratkd00@stud.fit.vut.cz>
00007 // $Date:       $2026-04-28
00008 //===========================================================================//
00009 #ifndef PARSER_H
00010 #define PARSER_H
00020 #include <stdbool.h>
00021
```

```
double parse_factorial(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_power(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_root(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_log(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_mul(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_div(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_add(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_sub(const char *equation, unsigned int lenght, unsigned int *error, bool *parsed);

double parse_equation(const char *equation, unsigned int lenght, unsigned int depth, unsigned int
*error);

double parse_number(const char *number, unsigned int lenght, unsigned int *error);
#endif // PARSER_H
```

2.11 test_math.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <math.h>
```

Include dependency graph for test_math.c:



Functions

- double add (double a, double b)
- double sub (double a, double b)
- double mul (double a, double b)
- double divide (double a, double b)
- long long factorial (int n)
- double power (double base, int exponent)
- double n_root (double degree, double radicand)
- double absolute (double a)
- double logarithm (double x)
- int is_equal (double a, double b)
- int main ()

2.11.1 Function Documentation

2.11.1.1 absolute()

```
double absolute (
            double a )
```

2.11.1.2 add()

```
double add (
            double a,
            double b )
```

2.11.1.3 divide()

```
double divide (
            double a,
            double b )
```

2.11.1.4 factorial()

```
long long factorial (
            int n )
```

2.11.1.5 is_equal()

```
int is_equal (
            double a,
            double b )
```

2.11.1.6 logarithm()

```
double logarithm (
            double x )
```

2.11.1.7 main()

```
int main ( )
```

2.11.1.8 mul()

```
double mul (
            double a,
            double b )
```

2.11.1.9 n_root()

```
double n_root (
            double degree,
            double radicand )
```

2.11.1.10 power()

```
double power (
            double base,
            int exponent )
```

2.11.1.11 sub()

```
double sub (
            double a,
            double b )
```

Index

absolute
test_math.c, 36
add
math.c, 10
math.h, 14
test_math.c, 36

divide
math.c, 10
math.h, 15
test_math.c, 36

err_msg
gui.c, 5

factorial
math.c, 11
math.h, 15
test_math.c, 36

gui.c, 3
err_msg, 5
main, 4
on_backspace_clicked, 4
on_button_clicked, 4
on_clear_clicked, 4
on_key_press, 5
run_calculator, 5
gui.h, 6
on_backspace_clicked, 7
on_button_clicked, 7
on_clear_clicked, 8
on_key_press, 8
run_calculator, 8

is_equal
test_math.c, 36

logarithm
math.c, 11
math.h, 15
test_math.c, 36

main
gui.c, 4
test_math.c, 36
math.c, 9
add, 10
divide, 10
factorial, 11
logarithm, 11
mul, 12
power, 12
root, 12
sub, 12
math.h, 13
add, 14
divide, 15
factorial, 15
logarithm, 15
mul, 16
n_root, 16
power, 16
sub, 17
mul
math.c, 12
math.h, 16
test_math.c, 36

n_root
math.h, 16
test_math.c, 36

on_backspace_clicked
gui.c, 4
gui.h, 7
on_button_clicked
gui.c, 4
gui.h, 7
on_clear_clicked
gui.c, 4
gui.h, 8
on_key_press
gui.c, 5
gui.h, 8

parse_add
parser.c, 20
parser.h, 30
parse_div
parser.c, 20
parser.h, 30
parse_equation
parser.c, 20
parser.h, 31
parse_factorial
parser.c, 21
parser.h, 31
parse_log
parser.c, 21
parser.h, 32

parse_mul
parser.c, 22
parser.h, 32
parse_number
parser.c, 22
parser.h, 33
parse_power
parser.c, 23
parser.h, 33
parse_root
parser.c, 23
parser.h, 33
parse_sub
parser.c, 23
parser.h, 34
parser.c, 18
parse_add, 20
parse_div, 20
parse_equation, 20
parse_factorial, 21
parse_log, 21
parse_mul, 22
parse_number, 22
parse_power, 23
parse_root, 23
parse_sub, 23
parser.h, 28
parse_add, 30
parse_div, 30
parse_equation, 31
parse_factorial, 31
parse_log, 32
parse_mul, 32
parse_number, 33
parse_power, 33
parse_root, 33
parse_sub, 34
power
math.c, 12
math.h, 16
test_math.c, 37

root
math.c, 12
run_calculator
gui.c, 5
gui.h, 8

sub
math.c, 12
math.h, 17
test_math.c, 37

test_math.c, 35
absolute, 36
add, 36
divide, 36
factorial, 36
is_equal, 36
logarithm, 36
main, 36
mul, 36
n_root, 36
power, 37
sub, 37